

DCTwin — dcwiz Template Compliance Report

Bottom line. DCTwin adopts *both* dcwiz standard policy templates: `RecommendTemplate` (subclassed by `WeeklyPlanTemplate` in `src/plan_weekly.py`) and `TrajectoryPolicyTemplate` (subclassed by `AITrajectoryReplay` in `src/ai_trajectory_test.py` and by `BaselineTrajectory` in `src/baseline_policy_test.py`). It mirrors the `sample_template/` layout for `configs/`, `data/`, and `models/` — the device-mapping JSON schemas (`device_key_map.json`, `device_his_map.json`, `room2ite_map.json`) and the `dt.prototxt/test.prototxt/train.prototxt` protobuf shapes are honored either identically or near-identically. The base-class contracts (`__init__` → `__call__` → `initialize` → `run`) are respected: the subclasses override `initialize()` and (for the planner) `run()` exactly where the templates designate those as the extension points. DCTwin **intentionally omits the RL-training artifacts** — `ai_policy_train.py`, the single-file `hooks.py`, the trained `policy/policy.pth`, and the `recommend.prototxt/train_test.prototxt` configs — because it is a *search/planning optimizer* (beam search over 3 setpoints scored by an EnergyPlus oracle), not a trained dcbrain RL policy. That functionality is not missing so much as relocated: reward/objective logic lives in `planner/objective.py` + `planner/kpi.py`, state discovery in `planner/monitor.py` + `planner/oracle_worker.py`, recommendation logging in `planner/recommendation.py`, zone→actuator mapping in `planner/broadcast.py`, and "model prep" in `fit_forecaster.py` (producing `forecaster.pkl` rather than `policy.pth`). One real defect surfaced: `AITrajectoryReplay` invokes `run(policy="ai")` without setting `self.test_collector`, which the template's `run_ai()` requires — a runtime incompatibility flagged below.

Compliance at a glance

Template artifact	DCTwin counterpart	Verdict
<code>RecommendTemplate</code> / <code>ai_policy_test.py</code>	<code>src/plan_weekly.py</code> → <code>WeeklyPlanTemplate(RecommendTemplate)</code>	adapted
<code>TrajectoryPolicyTemplate</code> / <code>ai_trajectory_test.py</code>	<code>src/ai_trajectory_test.py</code> → <code>AITrajectoryReplay(TrajectoryPolicyTemplate)</code>	adapted (partial — <code>test_collector</code> not set)
<code>baseline_policy_test.py</code>	<code>src/baseline_policy_test.py</code> → <code>BaselineTrajectory(TrajectoryPolicyTemplate)</code>	followed
<code>ai_policy_train.py</code>	none (closest analogue: <code>src/fit_forecaster.py</code>)	missing (by design)
<code>hooks.py</code> (single file)	distributed: <code>objective.py</code> , <code>kpi.py</code> , <code>monitor.py</code> , <code>oracle_worker.py</code> , <code>recommendation.py</code> , <code>broadcast.py</code> , <code>configs/dt/room2ite_map.json</code>	adapted
<code>configs/</code> (dt + policy)	<code>src/configs/</code>	adapted (omits <code>recommend.prototxt</code> , <code>train_test.prototxt</code>)
<code>data/</code> formats	<code>src/data/</code>	adapted (EPW + schedule JSON kept; adds calibration/realized-history)
<code>models/</code>	<code>src/models/</code>	extended (same <code>building.json</code> schema; IDF moved to <code>idf/</code> ; adds <code>forecaster.pkl</code>)

Template artifact	DCTwin counterpart	Verdict
<code>policy/</code> (<code>policy.pth</code>)	none — <code>BeamPlanner</code> (<code>planner/beam_search.py</code>) is the "policy"	missing (by design)

Legend: **followed** = template contract honored as-is; **adapted** = honored with deliberate, justified divergences; **partial** = contract partly honored with a gap; **missing** = artifact absent (here: by architectural design); **extended** = template honored plus domain-specific additions.

Side-by-side directory map

`sample_template/` (dcwiz canonical) vs `src/` (DCTwin). ✓ = match, + = DCTwin extension, ✗ = absent in DCTwin.

<code>sample_template/</code>	<code>src/</code> (DCTwin)	
<code>ai_policy_test.py</code>	<code>plan_weekly.py</code> (<code>WeeklyPlanTemplate</code>)	✓ adapted role
<code>ai_trajectory_test.py</code>	<code>ai_trajectory_test.py</code> (<code>AITrajectoryReplay</code>)	✓
<code>baseline_policy_test.py</code>	<code>baseline_policy_test.py</code> (<code>BaselineTrajectory</code>)	✓
<code>ai_policy_train.py</code>	—	✗ (no RL training)
<code>hooks.py</code>	— (logic split into <code>planner/</code> *)	✗ file / ✓ logic
<code>main_example.py</code>	—	✗
<code>configs/dt/dt.prototxt</code>	<code>configs/dt/dt.prototxt</code>	✓
<code>configs/dt/device_key_map.json</code>	<code>configs/dt/device_key_map.json</code>	✓
<code>configs/dt/device_his_map.json</code>	<code>configs/dt/device_his_map.json</code>	✓
<code>configs/dt/room2ite_map.json</code>	<code>configs/dt/room2ite_map.json</code>	✓
<code>configs/policy/test.prototxt</code>	<code>configs/policy/test.prototxt</code>	✓
<code>configs/policy/train.prototxt</code>	<code>configs/policy/train.prototxt</code>	✓
<code>configs/policy/recommend.prototxt</code>	—	✗
<code>configs/policy/train_test.prototxt</code>	—	✗
<code>data/weather/*.epw</code>	<code>data/weather/SGP_Singapore.486980_IWEC.epw</code>	✓ EPW format
<code>data/schedule/acus/</code>	<code>data/schedule/acus/fan_on_off/</code>	✓ (subfolder)
<code>data/schedule/branches/</code>	<code>data/schedule/branches/</code>	✓
<code>data/schedule/workloads/</code>	<code>data/schedule/workloads/</code>	✓
—	<code>data/schedule/pumps/</code>	+ (CHW plant)
<code>data/data_sample.json</code>	<code>data/his_data_processed.csv</code>	✓ adapted role
—	<code>data/calibration.json</code> , <code>calibration_history.json</code> , <code>realized_history.csv</code>	+ (dual-loop)
<code>models/building.json</code>	<code>models/building.json</code>	✓
<code>models/building.idf</code>	<code>models/idf/building.idf</code>	✓ (subfolder)

sample_template/	src/ (DCTwin)	
—	models/forecaster.pkl	+ (statistical forecaster)
policy/policy.pth	— (planner/beam_search.py::BeamPlanner)	X (algorithmic policy)

RecommendTemplate (AI policy)

Verdict: adapted. `WeeklyPlanTemplate` correctly subclasses `RecommendTemplate` and imports it from `dcwiz_policy_template`. It overrides `initialize()` and `run()` with batch-planner logic; the remaining base methods are inherited but partially bypassed for a deterministic planner.

The load-bearing subclass lines:

```
# src/plan_weekly.py:11
from dcwiz_policy_template import RecommendTemplate
# src/plan_weekly.py:22
class WeeklyPlanTemplate(RecommendTemplate):
```

Proof

Claim	Template ref	DCTwin ref	Evidence
Subclasses <code>RecommendTemplate</code> , imports from <code>dcwiz_policy_template</code>	<code>recommend_template.py</code> :10	<code>plan_weekly.py</code> :11,22	<code>class RecommendTemplate:</code> → <code>from dcwiz_policy_template import RecommendTemplate</code> and <code>class WeeklyPlanTemplate(RecommendTemplate)</code>
<code>initialize()</code> overridden with concrete impl	<code>recommend_template.py</code> :11	<code>plan_weekly.py</code> :30	Base raises <code>NotImplementedError</code> ; DCTwin fully implements forecaster, oracle, beam-planner setup
<code>run()</code> overridden for batch search	<code>recommend_template.py</code> :25-50	<code>plan_weekly.py</code> :67-86	Base calls <code>self.policy.policy(data)</code> ; DCTwin calls <code>run_weekly_plan()</code> with oracle evaluator + forecaster
<code>log_recommendations()</code> not overridden	<code>recommend_template.py</code> :60-72	ABSENT	Base defines the abstract method; DCTwin never overrides it (never reached — <code>run()</code> is fully replaced)
<code>configure_run_period()</code> not used	<code>recommend_template.py</code> :74-113	ABSENT	DCTwin docstring: it does NOT pass <code>recommendation_timestamp</code> to <code>__call__</code>
<code>_default_data()</code> bypassed	<code>recommend_template.py</code> :121-128	<code>plan_weekly.py</code> :62-65	<code>initialize()</code> comment: "satisfy base-class attribute presence (we override run, so these are unused)"
<code>__init__()</code> not overridden	<code>recommend_template.py</code> :52-58	<code>plan_weekly.py</code> :22-86	Only <code>initialize()</code> and <code>run()</code> defined; <code>__init__</code> inherited from base

Deviations. DCTwin adapts `RecommendTemplate` for a deterministic batch planner (beam search), not a reactive RL policy. `run()` is completely replaced to call `run_weekly_plan()` instead of `self.policy.policy(data)`; `log_recommendations()` is not implemented; `recommendation_timestamp` is excluded because the oracle controls the run period via `week_start`; `initialize()` requires a `week_start` kwarg for batch semantics. `__call__`, `configure_run_period`, `_default_data`, and `_init__` are inherited but partially bypassed.

TrajectoryPolicyTemplate (AI trajectory)

Verdict: adapted (partial). `AITrajectoryReplay` correctly subclasses `TrajectoryPolicyTemplate` and overrides `initialize()` to set `self.env` and `self.act` as the contract requires. However, it invokes `run(policy="ai")` without setting `self.test_collector`, which the template's `run_ai()` checks for and rejects — a runtime gap.

```
# src/ai_trajectory_test.py:14
class AITrajectoryReplay(TrajectoryPolicyTemplate):
# src/ai_trajectory_test.py:36
AITrajectoryReplay()(policy="ai", dt_engine_config=args.dt, recommendation=args.recommendation)
```

Proof

Claim	Template ref	DCTwin ref	Evidence
Subclasses <code>TrajectoryPolicyTemplate</code>	<code>trajectory_policy_template.py:15</code> ; <code>ai_trajectory_test.py:131</code> (canonical)	<code>ai_trajectory_test.py:14</code>	Canonical <code>TestAITrajectoryPolicy(TrajectoryPolicyTemplate)</code> ; DCTwin <code>AITrajectoryReplay(TrajectoryPolicyTemplate)</code>
Overrides <code>initialize()</code>	<code>trajectory_policy_template.py:614-627</code> (raises <code>NotImplementedError</code>)	<code>ai_trajectory_test.py:17-28</code>	DCTwin reads <code>dt_engine_config</code> from kwargs, builds env + mapped action
Sets <code>self.env</code>	<code>trajectory_policy_template.py:19</code> ; <code>run_ai:247</code>	<code>ai_trajectory_test.py:20</code>	<code>self.env = dctwin.make_env(env_proto_config=dt_engine_config, reward_fn=lambda x: 0)</code>
Sets <code>self.act</code> (fixed replay)	<code>trajectory_policy_template.py:20</code> ; <code>run_baseline:155</code>	<code>ai_trajectory_test.py:28</code>	<code>self.act = mapper_from_env(self.env).expand(setpoints)</code>
Invokes <code>run()</code> via <code>__call__</code> with <code>policy="ai"</code>	<code>trajectory_policy_template.py:110-117</code> ; <code>run_ai:207-338</code>	<code>ai_trajectory_test.py:36</code>	<code>AITrajectoryReplay()(policy="ai", ...)</code> dispatches to <code>run_ai()</code>
Uses <code>__call__</code> entry point (not <code>run()</code> directly)	<code>trajectory_policy_template.py:629-676</code>	<code>ai_trajectory_test.py:36</code>	Instance called as a callable; base <code>__call__</code> delegates to <code>initialize</code> + <code>run</code>
Does NOT override <code>run()/run_ai()</code>	<code>trajectory_policy_template.py:207-338</code>	no override	Template's <code>run_ai()</code> (custom_step wrapper, per-step temp collection) executes unchanged
Replay (fixed setpoints) vs live AI policy	<code>ai_trajectory_test.py:131-143</code> (canonical)	<code>ai_trajectory_test.py:14-28</code>	Canonical: <code>dcbrain.init(...) + test_collector.collect()</code> ; DCTwin: load recommendation JSON → fixed <code>self.act</code>

Claim	Template ref	DCTwin ref	Evidence
Does NOT set <code>self.test_collector</code>	<code>trajectory_policy_template.py:127-205</code> (<code>run_baseline</code> steps env directly)	<code>ai_trajectory_test.py:17-28</code>	DCTwin sets <code>self.env</code> , <code>self.act</code> but no <code>self.test_collector</code>
Year-split inherited but unused	<code>trajectory_policy_template.py:350-469, 629-676</code>	<code>ai_trajectory_test.py:36</code>	<code>recommendation_timestamp</code> not passed → <code>configure_run_period/_run_with_year_split_ai</code> bypassed
Fixed action maps to baseline pathway semantics	<code>trajectory_policy_template.py:119-205, 49-117</code>	<code>ai_trajectory_test.py:36</code>	<code>run_ai()</code> line 227 checks if <code>self.test_collector</code> is <code>None</code> : <code>raise ValueError("Test collector is not set")</code> — errors unless set, or <code>policy="baseline"</code>

Deviations.

- 1 **Missing `test_collector` (runtime incompatibility).** `AITrajectoryReplay` invokes `run(policy="ai")` but does not set `self.test_collector`; the template's `run_ai()` (lines 227–229) raises `ValueError("Test collector is not set")`. As written, this crashes when called with `policy="ai"`. The class should either set `self.test_collector` in `initialize()` or call with `policy="baseline"` (fixed-action replay matches baseline semantics — constant action per step, no AI inference).
- 2 **Replay vs live AI policy.** The canonical `TestAITrajectoryPolicy` uses `dcbrain.init()` to set up a live policy + `test_collector`, then `test_collector.collect()` generates actions dynamically. DCTwin loads a pre-computed recommendation JSON and expands it to a fixed action array — a valid "replay" adaptation, but it diverges from the template's assumption of a trained policy object.
- 3 **No `recommendation_timestamp/year-split` usage.** Line 36 does not pass `recommendation_timestamp`, so year-wrap splitting is never triggered. Acceptable for a single-week recommendation, but differs from the canonical example's arbitrary-date-range support.
- 4 **Lean overrides.** `AITrajectoryReplay` is ~14 lines with a single overridden method vs the canonical's heavier boilerplate — a deliberate simplification for a replay-only scope, not a contract violation.

Baseline trajectory test

Verdict: followed. `BaselineTrajectory` fully honors the `TrajectoryPolicyTemplate` contract for baseline trajectory testing: it subclasses correctly, implements `initialize()` to set the env and a conservative `self.act`, calls the base `run_baseline()` via `policy="baseline"`, and uses the same `__call__` entry point.

```
# src/baseline_policy_test.py:12
class BaselineTrajectory(TrajectoryPolicyTemplate):
# src/baseline_policy_test.py:27
BaselineTrajectory()(policy="baseline", dt_engine_config=args.dt)
```

Proof

Claim	Template ref	DCTwin ref	Evidence
Subclasses <code>TrajectoryPolicyTemplate</code>	<code>baseline_policy_test.py:32</code>	<code>baseline_policy_test.py:12</code>	<code>class TestBaselineProjectTemplate(TrajectoryPolicyTemplate) → class BaselineTrajectory(TrajectoryPolicyTemplate)</code>
Overrides <code>initialize()</code> with env setup	<code>baseline_policy_test.py:33-60</code>	<code>baseline_policy_test.py:15-20</code>	Both retrieve <code>dt_engine_config</code> from <code>kwargs</code> , call <code>dctwin.make_env()</code> , set <code>self.env + self.act</code>
Uses base <code>run_baseline</code> via <code>policy="baseline"</code>	<code>baseline_policy_test.py:95-99</code>	<code>baseline_policy_test.py:27</code>	Both invoke <code>__call__(policy="baseline", ...)</code> , routing to <code>run_baseline()</code> (<code>trajectory_policy_template.py:111-112</code>)
Baseline semantics: coolest SAT/CHW, max airflow	<code>baseline_policy_test.py:43-46</code>	<code>baseline_policy_test.py:13,19</code>	Template: <code>[-1]*16</code> (min SAT), <code>[0.9..1.0]</code> (max airflow), chiller <code>[-1]</code> . DCTwin: <code>sat_c=s.sat.lb, flow_kg_s=s.flow.ub, chwst_c=s.chwst.lb</code> from <code>DEFAULT_SEARCH_SPACE</code> (<code>sat=[20,26]</code> , <code>flow=[4.8,13.8]</code> , <code>chwst=[13,19]</code>)
Conservative action via <code>Setpoints → mapper_from_env().expand()</code>	<code>baseline_policy_test.py:42-60</code>	<code>baseline_policy_test.py:18-20</code>	Template hand-builds per-zone arrays; DCTwin builds <code>Setpoints(...)</code> then <code>mapper_from_env(self.env).expand(baseline)</code> (broadcast to all zones, <code>broadcast.py:46-56</code>)
Signature: <code>initialize</code> receives <code>**kwargs</code> with <code>dt_engine_config</code>	<code>baseline_policy_test.py:33-36</code>	<code>baseline_policy_test.py:15-16</code>	Both <code>kwargs.get("dt_engine_config", ...)</code> with the same default fallback
<code>run_baseline()</code> not overridden; base method runs	<code>trajectory_policy_template.py:110-126</code>	<code>baseline_policy_test.py:12-20</code>	Base <code>run_baseline()</code> (lines 119-205) steps <code>self.act</code> to done, collects temps, writes CSV — matches contract exactly
Entry point: instantiate + call with <code>policy="baseline"</code>	<code>baseline_policy_test.py:92-100</code>	<code>baseline_policy_test.py:23-28</code>	Identical <code>__call__</code> invocation pattern

Deviations. Only a benign encoding difference: the template hard-codes conservative setpoints as normalized per-zone action arrays; DCTwin parametrizes via `Setpoints` + dynamic `BroadcastPolicy.expand()`. The physical semantics (coolest SAT/CHW, maximum airflow → safe, energy-heavy) are identical. No functionality gaps or contract violations.

AI policy train

Verdict: missing (by design). DCTwin has no `ai_policy_train.py` and no `src/policy/` directory because it is not an RL policy learner. It performs one-shot heuristic search (`BeamPlanner`) scored by an EnergyPlus oracle, rather than training a dcbain SAC policy via reward-driven episodes.

Proof

Claim	Template ref	DCTwin ref	Evidence
Template trains a dcbain RL policy → <code>policy/policy.pth</code>	<code>ai_policy_train.py:10-16</code>	ABSENT	Template: <code>policy, train_collector, test_collector, trainer = dcbain.init(...); trainer.run() → policy/policy.pth</code> (2.9 MB). DCTwin has no <code>ai_policy_train.py</code> ; <code>src/policy/</code> does not exist
Template uses <code>train.prototxt</code> (SAC, 20 epochs, batch 256)	<code>configs/policy/train.prototxt:7-32</code>	<code>src/configs/policy/train.prototxt:7-32</code>	Template: SAC, <code>max_epoch:20, batch_size:256</code> . DCTwin: SAC config present but <code>batch_size:512, step_per_epoch:5376</code> — config exists yet is NOT used (no <code>dcbain.init</code> call anywhere)
Template calls <code>hooks.get_reward_fn()</code> / <code>save_best_fn()</code>	<code>ai_policy_train.py:3, 8,13</code>	ABSENT from <code>src/</code>	Template: <code>from hooks import get_reward_fn, save_best_fn and make_env(..., reward_fn=get_reward_fn())</code> . No <code>hooks.py</code> in <code>src/</code> (<code>test/hooks.py</code> is for co-sim, not RL); no <code>reward_fn</code> or training loop in <code>src/</code>
DCTwin is a search/planning optimizer	n/a	<code>plan_weekly.py:22-26; planner/beam_search.py:1-50</code>	<code>WeeklyPlanTemplate</code> docstring: "One-shot weekly planner: heuristic search over 3 setpoints, EnergyPlus-scored. Overrides <code>run()</code> because the base <code>RecommendTemplate.run()</code> expects a reactive dcbain policy." <code>BeamPlanner</code> = grid + beam refine with oracle, not gradient RL
<code>fit_forecaster.py</code> is the analogous "model prep"	<code>ai_policy_train.py:7-16</code> (RL prep → <code>policy.pth</code>)	<code>src/fit_forecaster.py:42-68</code>	DCTwin fits a forecaster config (persistence forecast + room→IT-load mapping) to a pickle: <code>main() → models/forecaster.pkl</code> (config/data prep, no learning loop)

Deviations. Fundamental architectural difference. The template trains a `policy.pth` neural network via `dcbain.init()` + `trainer.run()`. DCTwin pre-fits a forecaster config and uses deterministic search at runtime — no policy gradient, no episode collection, no trainer loop. Not a

violation; a deliberate pivot from learned control to search-based planning.

Hooks

Verdict: adapted. There is no single `hooks.py` in `src/`. The hook responsibilities are distributed across the `planner/` package and `configs/dt/room2ite_map.json`. The `RecommendTemplate` contract (`initialize` + `run` + `log_recommendations`) is honored; the per-step RL hooks are superseded by the oracle-based evaluation model.

Proof

Claim (template hook)	Template ref	DCTwin ref	Evidence
<code>zone_acu_dict</code> (zone→device map)	<code>hooks.py:19-133</code>	<code>configs/dt/room2ite_map.json:1-50</code>	Template Python dict; DCTwin JSON config maps "Data Hall 1F 2A" → ITE resources — same role for thermal KPI scoping
<code>get_reward_fn()</code> (power/temp/RH penalties)	<code>hooks.py:149-241</code>	<code>planner/objective.py:47-67</code> , <code>planner/kpi.py:50-118</code>	Template: PUE/HVAC/inlet/chiller reward terms. DCTwin: <code>objective.score() = energy_term + λ_temp·inlet_excess + λ_rh·rh_excursion + λ_zone·zone_band</code> ; <code>kpi.aggregate_kpi</code> computes <code>weighted_energy_cost</code> , <code>inlet_violation_steps</code> , <code>rh_violation_steps</code> , <code>zone_temp_band_steps</code>
<code>get_current_state(env, data_file, obs_keys)</code>	<code>hooks.py:339-355</code>	<code>planner/oracle_worker.py:32-46</code> , <code>planner/monitor.py:35-71</code>	Template reads JSON, returns <code>[1, n_obs]</code> array. DCTwin: <code>read_step_sample()</code> calls <code>env.inspect_current_observation()</code> over <code>MonitorSpec</code> -discovered names → <code>StepSample</code> ; <code>discover_monitor()</code> auto-detects obs names
<code>log_recommendations(env, data, data_file)</code>	<code>hooks.py:358-392</code>	<code>planner/recommendation.py:25-160</code>	Template writes per-action JSON. DCTwin: <code>build_recommendation()</code> → setpoints + predicted KPIs + forecast metadata + robust diagnostics; <code>write_recommendation()</code> serializes
<code>get_reward_fn_single_hall(target_hall)</code>	<code>hooks.py:248-329</code>	<code>planner/oracle.py:33-34</code> , <code>planner/oracle_worker.py:17-29</code>	Template filters <code>zone_acu_dict[target_hall]</code> . DCTwin: <code>OracleConfig.monitored_hall="1f 2a"</code> , <code>EvalTask.monitored_hall</code> scopes thermal KPI to the controlled hall

Claim (template hook)	Template ref	DCTwin ref	Evidence
<code>save_best_fn(policy)</code>	<code>hooks.py:244-246</code>	ABSENT	Template <code>torch.save(policy.state_dict(), policy.pth)</code> . DCTwin is a search optimizer — no policy artifact; output is <code>recommendation.json</code>
<code>zone_names</code> list	<code>hooks.py:135-139</code>	implicit in <code>room2ite_map.json</code> keys	Template <code>['DH_1', 'DH_2', 'DH_4']</code> ; DCTwin: zone identifiers are the JSON keys (no explicit list)
<code>zone_acu_model_map</code>	<code>hooks.py:142-146</code>	<code>planner/plant.py</code> + <code>broadcast.py</code>	DCTwin: <code>BroadcastPolicy</code> maps 3 globals → N actuators via <code>ActionEntry</code> kinds; <code>plant.py</code> models chiller/CHW energy
device-name regex normalization	<code>hooks.py:16</code>	<code>planner/env_actions.py</code> / <code>broadcast.py</code>	DCTwin works with actuator names in DT-prototxt declaration order; consistent naming via prototxt rather than regex
No <code>ai_policy_train.py</code>	<code>ai_policy_train.py</code> (exists)	ABSENT	DCTwin is search-based; no RL training loop, no <code>policy.pth</code>
No <code>hooks.py</code> in <code>src/</code>	<code>sample_template/hooks.py</code>	ABSENT from <code>src/</code>	<code>find src -name hooks.py</code> → none (<code>test/hooks.py</code> , <code>tutorials/co-sim/hooks.py</code> are EnergyPlus-adaptor, not policy templates)

Deviations. The hook functionality is reorganized, not omitted: reward → `objective.py` + `kpi.py`; state observation → `oracle_worker.py` + `monitor.py`; recommendation logging → `recommendation.py`; zone config → `room2ite_map.json`. Reward semantics differ in kind: the template's `reward_fn` returns a per-step scalar for RL gradient; DCTwin's `objective.score()` is a weekly rollup (energy + soft penalties) used to rank beam-search candidates. `save_best_fn/policy.pth` are N/A because there is no learned policy.

Configs

Verdict: adapted. DCTwin keeps the `configs/dt/` file set and JSON schemas identical, and keeps `test.prototxt/train.prototxt/dt.prototxt` protobuf shapes identical. It deliberately omits the two RL-only policy configs.

Proof

Claim	Template ref	DCTwin ref	Evidence
<code>dt/</code> file set matches (4 files)	<code>configs/dt/</code>	<code>configs/dt/</code>	Both: <code>dt.prototxt</code> , <code>device_his_map.json</code> , <code>device_key_map.json</code> , <code>room2ite_map.json</code>

Claim	Template ref	DCTwin ref	Evidence
device_key_map.json top-level schema	device_key_map.json (13 keys)	device_key_map.json (13 keys)	Identical set of 13 top-level keys (acus, chilled water loops, chilled water pumps, chillers, condenser water loops, condenser water pumps, cooling towers, dehumidifiers, heat_exchangers, ites, secondary chilled water pumps, thermal storage tanks, zones). Same schema, fewer device instances: template 2814 lines vs DCTwin 918
device_key_map.json zones schema (3 sub-keys)	device_key_map.json:1 8-113	device_key_map.json:1 8-54	Both: {air temperature, air relative humidity, ite power}
device_key_map.json ites schema (5 sub-keys)	device_key_map.json:1 15-158	device_key_map.json:5 5-62	Both: {inlet dry-bulb temperature, inlet relative humidity, cpu power, fan power, ups power}
device_his_map.json top-level matches	device_his_map.json:1 -30	device_his_map.json:1 -27	Identical structure (chilled/condenser water loops, zones, ites, ...)
room2ite_map.json schema matches	room2ite_map.json:1-3 0	room2ite_map.json:1-3 0	Both: {room → {ite → {wattsPerUnit, numberOfUnits, totalWatts}}}
test.prototxt structure	configs/policy/test.p rototxt:1-27	configs/policy/test.p rototxt:1-27	Both: logging_config, policy_config, test_collector_config at the same lines
train.prototxt structure	configs/policy/train. prototxt:1-30	configs/policy/train. prototxt:1-30	Both: logging_config, policy_config, off_policy_trainer_config
dt.prototxt eplus_env_config shape	configs/dt/dt.prototx t:1-10	configs/dt/dt.prototx t:1-10	Both: eplus_env_config { weather_file, model_file, simulation_time_config, ... }
Omits recommend.prototxt	configs/policy/recomm end.prototxt (present)	ABSENT	DCTwin does not run inference on pre-trained RL policies
Omits train_test.prototxt	configs/policy/train_ test.prototxt (present)	ABSENT	DCTwin does not use combined RL train+test

Deviations. `recommend.prototxt` (inference on a trained RL policy) and `train_test.prototxt` (combined RL train+test) are intentionally absent — DCTwin is a search/planning optimizer, not an RL system. All shared JSON schemas and the `test/train/dt` protobuf shapes are honored precisely.

Data formats

Verdict: adapted. DCTwin follows the schedule-folder convention (`acus/`, `branches/`, `workloads/` with JSON arrays) and the EPW weather standard, replaces the static `data_sample.json` with continuous per-15-minute telemetry, and extends `data/` with calibration + realized-history files for

dual-loop learning.

Proof

Claim	Template ref	DCTwin ref	Evidence
<code>schedule/</code> subfolders follow template (+ <code>pumps/</code>)	<code>schedule/{acus,branches,workloads}/</code>	<code>schedule/{acus,branches,pumps,workloads}/</code>	Template: 3 dirs; DCTwin: 4 dirs. Template <code>acus/</code> : 48 files (<code>cra*_crac_x_.json</code>); DCTwin <code>acus/</code> : <code>fan_on_off/</code> with 22 files (<code>data hall * acu-*.json</code>)
<code>weather/</code> uses EPW on both sides	<code>weather/de_berlin.epw</code> (LOCATION,BERLIN,-,DEU,IWEC...)	<code>weather/SGP_Singapore.486980_IWEC.epw</code> (LOCATION,SINGAPORE,-,SGP,IWEC...)	Identical ASHRAE EPW header format
Workload files: JSON arrays of floats	<code>workloads/cra_1_07_crac_x_402.json:1-20</code>	<code>workloads/data hall 1f 2a ite-1.json:1-20</code>	Both are JSON arrays of per-interval floats
Branch files: JSON arrays	<code>branches/ (chilled water supply branch N.json)</code>	<code>branches/</code> (same naming, 5 files)	Same naming convention
ACU files: JSON arrays	<code>acus/cra_1_07_crac_x_401.json:1-3</code> ([1.0,1.0,...])	<code>acus/fan_on_off/data hall 1f 2a acu-1.json:1-30</code> ([1,1,1,...])	Both JSON on/off-schedule arrays
<code>data_sample.json</code> → <code>his_data_processed.csv</code>	<code>data/data_sample.json</code> :2-4 (single timestamped config dict)	<code>data/his_data_processed.csv</code> (<code>_time,ACLF,PUE,WCLF,..._Power</code>)	Shift from static policy sample to continuous per-15-min telemetry (100+ PDU power columns)
DCTwin EXTENDS <code>data/</code> with calibration/realized history	<code>data/</code> (none)	<code>data/calibration.json, calibration_history.json, realized_history.csv</code>	Adds bias/sigma dicts, paired predicted/realized weeks, weekly KPI records (<code>total_hvac_energy_kwh, pue_mean, inlet_temp_max_c</code>)
Calibration history stores paired (predicted, realized)	n/a (not in template)	<code>data/calibration_history.json</code> (<code>planner/history.py:46-60</code>)	<code>advance_calibration()</code> appends { <code>week_start, predicted, realized</code> } → dual-loop learning
Realized history CSV separate from forecaster CSV	n/a	<code>his_data_processed.csv</code> vs <code>realized_history.csv</code> (<code>planner/history.py:6-7,16-29</code>)	Comment: realized-KPI history must NOT be the forecaster's per-step IT-load CSV (different schema: 15-min, 384 columns)

Deviations. Domain-specific, not schema violations: (1) ACU schedules live under `fan_on_off/` with building-topology naming (`data hall * acu-*.json`) instead of generic `cra*_crac_x_.json`; (2) static `data_sample.json` → per-15-min `his_data_processed.csv` for forecasting; (3) `calibration.json/calibration_history.json/realized_history.csv` added for the P1-policy + P2-calibrator dual loop (absent from the RL baseline); (4) `pumps/` is a CHW-plant extension. The schedule-subfolder convention, EPW format, and JSON-array formats are all honored.

Models

Verdict: extended. `models/building.json` is schema-compliant (identical top-level keys); the IDF is moved into a `models/idf/` subfolder; a statistical `forecaster.pkl` is added in place of trained neural weights.

Proof

Claim	Template ref	DCTwin ref	Evidence
<code>building.json</code> structure matches	<code>models/building.json: 1-5</code>	<code>models/building.json: 1-5</code>	Identical top-level keys [<code>constructions</code> , <code>geometry</code> , <code>inputs</code> , <code>meta</code> , <code>models</code>]; <code>meta</code> has <code>name/description</code> , <code>models</code> has <code>curve models</code>
<code>building.idf</code> moved to <code>models/idf/</code>	<code>models/building.idf</code>	<code>models/idf/building.idf</code>	Template: <code>building.idf</code> in root (1207 KB). DCTwin: <code>idf/</code> subfolder, <code>idf/building.idf</code> (842 KB) — logical grouping
Adds <code>forecaster.pkl</code> (statistical, not neural)	n/a (template has no forecaster)	<code>models/forecaster.pkl</code>	<code>plan_weekly.py:40</code> loads it via pickle; <code>fit_forecaster.py:45-68</code> saves config (<code>method="persistence"</code> , <code>his_csv</code> , <code>room2ite_path</code> , <code>weather_file</code>). Template <code>policy.pth</code> (2.9 MB PyTorch) vs DCTwin <code>forecaster.pkl</code> (655 B config dict)
<code>WeeklyPlanTemplate</code> honors <code>RecommendTemplate.__call__</code>	<code>recommend_template.py:130-142</code>	<code>plan_weekly.py:22,30,67</code>	Base <code>__call__</code> runs <code>initialize()</code> , <code>run()</code> , <code>cleanup()</code> ; DCTwin imports <code>RecommendTemplate</code> (l.11), defines <code>initialize</code> (l.30) + <code>run</code> (l.67), <code>run()</code> overridden to call <code>run_weekly_plan()</code> (l.73-81)
Trajectory subclasses honor <code>TrajectoryPolicyTemplate</code>	<code>trajectory_policy_template.py:15,629</code>	<code>ai_trajectory_test.py:14,17</code> ; <code>baseline_policy_test.py:12,15</code>	Both subclass <code>TrajectoryPolicyTemplate</code> and define <code>initialize()</code> ; both use the <code>__call__(policy=...)</code> protocol
<code>src/policy/</code> ABSENT (no <code>policy.pth</code>)	<code>policy/policy.pth</code>	ABSENT	DCTwin's "policy" is <code>BeamPlanner</code> (<code>beam_search.py</code>); <code>plan_weekly.py:61</code> assigns <code>self.planner = BeamPlanner(...)</code> to <code>self.policy</code> (l.63) as a base-class stub; <code>run()</code> overrides the policy call entirely

Claim	Template ref	DCTwin ref	Evidence
<code>run()</code> override skips <code>log_recommendations</code>	<code>recommend_template.py:25-50,60-72</code>	<code>plan_weekly.py:67-86</code>	Base <code>run()</code> calls <code>self.log_recommendations()</code> (would raise <code>NotImplementedError</code>); DCTwin <code>run()</code> fully overrides, calls <code>run_weekly_plan() + write_recommendation()</code> directly, never reaches <code>log_recommendations()</code>
No <code>hooks.py</code> / <code>ai_policy_train.py</code>	<code>hooks.py:1-242; ai_policy_train.py:1-24</code>	ABSENT	No RL reward infra, no <code>dcbrain.init + trainer.run()</code> ; replaced by <code>fit_forecaster.py + planner/beam_search.py</code>
<code>configs/policy/</code> missing <code>recommend/train_test</code> prototxt	<code>configs/policy/{recommend,test,train,train_test}.prototxt</code>	<code>configs/policy/{test,train}.prototxt</code> only	DCTwin has 2 of 4 policy prototxts

Deviations. Intentional, by architecture: (1) IDF reorganized to `models/idf/`; (2) `forecaster.pkl` (655 B statistical config) replaces `policy.pth` (2.9 MB neural weights); (3) no `policy/` directory — the policy is the in-code `BeamPlanner`; (4) no `hooks.py/ai_policy_train.py` — replaced by `fit_forecaster.py + planner/`; (5) `configs/policy/` omits the RL-only prototxts; (6) `WeeklyPlanTemplate.run()` override is the intended extension point, so `log_recommendations()` is never called. All `RecommendTemplate/TrajectoryPolicyTemplate` base-class contracts for `__call__`, `initialize`, `run` are honored.

Policy

Verdict: missing (by design). There is no `src/policy/` and no `policy.pth`. DCTwin's optimization "policy" is the algorithmic `BeamPlanner` (`planner/beam_search.py`), an in-code heuristic search over the setpoint grid — not a trained weights artifact.

Proof

Claim	Template ref	DCTwin ref	Evidence
Template <code>policy/</code> holds trained <code>policy.pth</code>	<code>policy/policy.pth</code>	ABSENT	Template: 2.9 MB trained RL policy from <code>dcbrain</code> . DCTwin: no <code>policy/</code> dir
DCTwin's "policy" is <code>BeamPlanner</code> (algorithmic)	n/a	<code>plan_weekly.py:61,63; planner/beam_search.py</code>	<code>self.planner = BeamPlanner(...)</code> assigned to <code>self.policy</code> as a base-class stub; deterministic enumeration + beam refine, never invoked through the base <code>run()</code> contract
Beam search configured in Python, not prototxt	<code>configs/policy/*.prototxt</code>	<code>plan_weekly.py:56-60</code> (<code>BeamConfig</code> dataclass)	DCTwin configures search via a <code>BeamConfig</code> dataclass, not protobuf

Deviations. No learned weights exist to persist. The `policy/policy.pth` artifact and its surrounding RL machinery are deliberately replaced by deterministic optimization. The base-class attribute `self.policy` is still satisfied (it points at the `BeamPlanner` stub), so the template's structural contract holds even though the storage artifact is absent.

Where DCTwin diverges (and why)

The single architectural fact explains every "missing" verdict: **DCTwin is a search/planning optimizer, not a trained RL agent**. The dcwiz sample template assumes a dcbrain SAC policy — a neural network trained on the dctwin environment via reward-driven episodes, persisted as `policy/policy.pth`, fed per-step rewards and states through `hooks.py`, and trained by `ai_policy_train.py` (`dcbrain.init()` + `trainer.run()`). DCTwin instead searches three global setpoints (SAT, airflow, CHWST) with a beam search whose every candidate is scored by a full-week EnergyPlus oracle run.

That swap removes the entire RL training pipeline and relocates its responsibilities:

Template (RL) artifact	DCTwin replacement	Where it lives
<code>policy/policy.pth</code> (trained weights)	<code>BeamPlanner</code> (in-code algorithm)	<code>planner/beam_search.py</code>
<code>ai_policy_train.py</code> (<code>trainer.run()</code>)	one-shot model/config prep	<code>fit_forecaster.py</code> → <code>models/forecaster.pkl</code>
<code>hooks.get_reward_fn()</code> (per-step scalar)	weekly objective rollup	<code>planner/objective.py</code> + <code>planner/kpi.py</code>
<code>hooks.get_current_state()</code>	observation discovery + sampling	<code>planner/monitor.py</code> + <code>planner/oracle_worker.py</code>
<code>hooks.log_recommendations()</code>	recommendation builder/serializer	<code>planner/recommendation.py</code>
<code>hooks.zone_acu_dict</code> / <code>zone_acu_model_map</code>	global→actuator broadcast + plant model	<code>planner/broadcast.py</code> + <code>planner/plant.py</code> + <code>configs/dt/room2ite_map.json</code>
<code>recommend.prototxt</code> / <code>train_test.prototxt</code>	Python <code>BeamConfig</code> dataclass	<code>plan_weekly.py</code> :56-60

So the reward function, state extraction, recommendation logging, and zone mapping all still exist — just as Python modules in `planner/` and JSON config, rather than as the template's single `hooks.py`. The forecaster (`fit_forecaster.py` → `forecaster.pkl`, a small statistical config) is DCTwin's "model preparation" step in the slot the template fills with neural-policy training.

Conclusion

Yes — with caveats. DCTwin did follow the dcwiz standard policy templates where they apply to its architecture:

- **RecommendTemplate: followed via adaptation.** `WeeklyPlanTemplate(RecommendTemplate)` imports from `dcwiz_policy_template`, overrides `initialize()` and `run()` at the designated extension points, and respects the `__init__`/`__call__` contract.
- **TrajectoryPolicyTemplate: followed for baseline, partially for AI replay.** `BaselineTrajectory` honors the contract end-to-end (subclass, `initialize()`, base `run_baseline()` via `policy="baseline"`). `AITrajectoryReplay` honors the structure and `initialize()` override but has a real gap — it calls `run(policy="ai")` without setting `self.test_collector`, which the template's `run_ai()` rejects at runtime. **Fix:** set `self.test_collector` in `initialize()`, or call with `policy="baseline"` (the fixed-action replay is baseline-shaped anyway).
- **configs/, data/, models/: honored.** JSON device-map schemas, EPW weather format, schedule-folder convention, and `dt/test/train` prototxt shapes all match; DCTwin adds domain-specific extensions (calibration/realized-history, `pumps/`, `forecaster.pkl`) without breaking the shared schemas.

- **RL-training artifacts** (`ai_policy_train.py`, **single-file** `hooks.py`, `policy/policy.pth`, `recommend.prototxt`, `train_test.prototxt`): **intentionally absent**. These are RL-pipeline-specific; DCTwin is a deterministic planner and replaces them with `fit_forecaster.py`, the `planner/` package, and `BeamPlanner`.

Net: the template *contracts* (base-class subclassing, the `initialize/run/__call__` lifecycle, the config/data/model layouts) are honored. The only outright contract gap to remediate is the `AITrajectoryReplay test_collector` omission; everything else is a justified architectural adaptation from "trained RL policy" to "search-based planner."